Identifying Large Language Models from Their Responses

CMPSC 448 — Machine Learning Research Project

Name: Min Sung Kim
Date: October 11, 2026

Introduction

Large Language Models (LLMs) such as GPT, Claude, and Gemini can answer questions, write stories, explain code, and solve mathematical problems. Although these systems often generate similar information when given the same prompt, their responses may differ in length, vocabulary, sentence structure, and formatting.

The main objective of this project was to compare Convolutional Neural Network (CNN) and Long Short-Term Memory (LSTM) architectures for classifying LLM-generated responses. I also wanted to determine whether knowing the original prompt improves classification accuracy and whether models trained on certain task categories can generalize to categories they have never encountered.

RQ1: Can CNN and LSTM classifiers identify which of three LLMs generated a given response?

RQ2: How does classification performance change when using the prompt alone, the response alone, or both together?

RQ3: Can a classifier trained on four task categories identify LLM responses from a fifth, unseen category?

RQ4: What measurable linguistic and formatting characteristics distinguish the responses generated by the three LLMs?

I initially conducted a pilot experiment using 300 responses. After examining the preliminary results, I expanded the dataset to 3,300 responses to investigate whether additional training examples would improve performance.

Dataset Collection and Preparation

Dataset Collection

GPT, Claude, and Gemini were used in this experiment. Each model received the same set of prompts, and its generated responses were stored together with the original prompt and the model label. The dataset contains three primary fields used for classification: `LLM_name`, `LLM_Input`, and `LLM_output`. Additional metadata, including prompt identifiers and categories, was used for organizing the experiments.

I designed prompts covering five different types of tasks:

Category	Description	Number of prompts
General Knowledge	Explanations, comparisons, and factual reasoning	220
Creative Writing	Stories, poetry, dialogue, and imaginative scenarios	220
Programming	Code generation, debugging, algorithms, and software concepts	220
Mathematical/Logical Reasoning	Mathematics, statistics, and logical problem-solving	220
Text Transformation	Rewriting, editing, summarization, and formatting	220
Total		**1,100**

The prompts also varied in their level of constraints. Some were open-ended, while others specified requirements such as response format, length, or content. This design allowed me to compare writing patterns across both structured and open-ended tasks.

Rationale for Selecting Five Prompt Categories

I selected the five prompt categories based on previous research about machine-generated text detection and LLM authorship attribution. One important challenge identified in this research is that classification models may perform well on familiar types of text but struggle when the writing domain or prompting method changes.

Wang et al. (2024) introduced M4GT-Bench, a benchmark for evaluating machine-generated text detection across multiple languages, domains, and text generators. Their research found that strong detection performance often depends on having training data from the same domain and generators as the evaluation data. The benchmark also includes multi-way detection, where the goal is to identify the model that generated a particular text.

Based on the study, I wanted my dataset to include multiple types of writing rather than focusing on one particular task. I selected five categories because each one represents a different type of language generation and may reveal different writing characteristics.

General Knowledge: I selected this category because factual questions and explanatory tasks allow me to examine how LLMs organize information and explain concepts. These responses often include explanations, comparisons, and structured presentation of information.

Creative Writing: This category includes stories, poetry, dialogue, and imaginative scenarios. I selected it because creative writing allows more freedom in vocabulary, sentence structure, and narrative style. It provides a contrast to factual and technical responses, which are often more structured.

Programming: This category contains code generation, debugging, algorithm explanations, and software-related questions. I selected it because programming responses often combine natural language with source code, technical terminology, and Markdown formatting. These features may produce recognizable differences between LLMs.

Mathematical/Logical Reasoning: This category includes mathematical problems, probability questions, and logical reasoning tasks. I selected it to examine how LLMs present calculations, organize explanations, and communicate step-by-step solutions.

Text Transformation: This category includes rewriting, paraphrasing, editing, and formatting tasks. I included it because previous research, particularly Beigi et al. (2024), examined rewriting and paraphrasing as distinct prompting methods for model attribution. These tasks are also different from open-ended generation because the output is constrained by existing source text.

The five categories were my own experimental design choice rather than a fixed category system required by previous research. I chose them to cover different writing styles while keeping the dataset manageable and balanced. Each category contained 220 prompts, and all three LLMs answered the same prompts. This allowed me to compare different LLMs under the same task conditions.

The category design was particularly important for RQ3. By training the classifiers on four categories and evaluating them on the remaining category, I could investigate whether the models learned writing characteristics that generalize across tasks or mainly depended on patterns associated with familiar categories.

Dataset Expansion

The first version of the dataset contained 100 prompts, with 20 prompts per category. Each prompt was submitted to all three LLMs, resulting in 300 responses.

After completing the initial experiments, I generated an additional 1,000 prompts and collected 3,000 new responses.

The final dataset contained:

Property	Value
Total prompts	1,100

Total responses	3,300
LLM classes	3
Responses per LLM	1,100
Task categories	5
Prompts per category	220
Responses per category	660

All 3,300 responses passed the project's dataset validation checks.

Data Preprocessing

Before training, the text was tokenized and converted into integer sequences. The preprocessing pipeline used lowercased tokens and recognized words, punctuation, and a special separator token for combined prompt-and-response input.

The vocabulary was constructed from the training data only, which prevented vocabulary information from the validation and test sets from being incorporated into training.

Each input sequence was limited to a maximum of 512 tokens. Longer sequences were truncated, while shorter sequences were padded as needed.

This provided a consistent input representation across experiments. However, truncating longer responses could remove information potentially useful for identifying the generating LLM.

Machine Learning Methodology

Training Configuration

Both CNN and LSTM architectures used the same general training settings.

Parameter	Value
Embedding dimension	64
Maximum sequence length	512
Dropout	0.3
Learning rate	0.001
Batch size	16
Maximum epochs	20
Early-stopping patience	4
Random seed	42
Checkpoint selection	Validation loss; minimum improvement of 0.0001

Training was conducted using PyTorch on a Mac with Apple's Metal Performance Shaders (MPS) backend.

Checkpoint selection used validation loss with a minimum improvement threshold of 0.0001. A new checkpoint was saved only when validation loss improved by at least that amount.

Therefore, the selected checkpoint was not necessarily the epoch with the mathematically lowest recorded validation loss. Early stopping was used to limit unnecessary training when validation loss stopped improving sufficiently.

Evaluation Metrics

I used two evaluation metrics: accuracy and macro F1-score. Accuracy measures the proportion of correctly classified responses. Accuracy = Correct Predictions / Total Predictions Macro F1-**score** calculates the F1-score separately for each LLM class and then averages the three values. Macro F1-score is useful because it indicates whether performance is consistent across GPT, Claude, and Gemini rather than being dominated by one class. Since each model contributed the same number of responses, the random-guessing baseline was approximately 33.3%.

Experiment 1: Identifying LLMs from Their Responses (RQ1)

Experimental Design

The first experiment evaluated whether CNN and LSTM could identify the generating LLM using only the generated response.

The expanded dataset was split by prompt ID, ensuring that responses to the same prompt did not appear in different partitions.

Partition	**Prompts**	**Responses**
Training	770	2,310
Validation	165	495
Test	165	495
Total	**1,100**	**3,300**

Each category contributed 154 training prompts, 33 validation prompts, and 33 test prompts.

Results

Architecture	**Test accuracy**	**Macro F1-score**
CNN	**86.7%**	**0.867**
LSTM	65.3%	0.645

The CNN achieved substantially better performance than the LSTM, exceeding it by approximately 21.4 percentage points in test accuracy.

Both models performed considerably better than random guessing, indicating that the generated responses contained information useful for distinguishing the three LLMs.

The CNN may have benefited from its ability to recognize local lexical and formatting patterns. However, I did not directly inspect the learned convolutional filters, so this remains a possible explanation rather than a confirmed mechanism.

Comparison with the Pilot Study

I also compared the expanded results with the original 300-response experiment.

Architecture	Pilot accuracy	Expanded accuracy	Change
CNN	73.3%	86.7%	+13.4 percentage points
LSTM	40.0%	65.3%	+25.3 percentage points

Both architectures achieved higher accuracy on the expanded dataset.

The LSTM showed the larger improvement, suggesting that its initial performance may have been limited partly by the small amount of training data.

Nevertheless, the two experiments used different test-set compositions, so the improvement cannot be attributed exclusively to increased dataset size.

Experiment 2: Comparing Input Configurations (RQ2)

Experimental Design

The second experiment examined which parts of an LLM interaction are useful for identifying the responding model.

I tested three configurations:

1. **Output only:** The classifier received only the generated response.
2. **Input only:** The classifier received only the original prompt.
3. **Input + output:** The classifier received the prompt followed by a separator and the generated response.

I used the same CNN and LSTM architectures and the same training, validation, and test split.

Results

Architecture	Input configuration	Test accuracy	Macro F1
CNN	Output only	86.7%	0.867
LSTM	Output only	65.3%	0.645
CNN	Input only	33.3%	0.271

LSTM	Input only	33.3%	0.285
CNN	Input + output	**88.7%**	**0.888**
LSTM	Input + output	64.4%	0.643

Discussion

Both architectures achieved exactly 33.3% accuracy using prompts alone. This outcome was expected because every prompt was submitted to all three LLMs. Therefore, the same prompt text was associated with three different labels and could not independently identify which LLM produced a response. For CNN, including both the prompt and response increased test accuracy from 86.7% to 88.7%. For LSTM, including the prompt slightly reduced accuracy from 65.3% to 64.4%. These results suggest that the generated response contains the main information needed for classification. The prompt may provide useful context for the CNN, but the observed improvement was relatively small. I cannot conclude that adding prompts consistently improves accuracy without repeated experiments or additional statistical analysis.

Experiment 3: Generalization to Unseen Task Categories (RQ3)

Experimental Design

The third experiment investigated whether authorship-identification models could generalize beyond the categories they encountered during training. I used a leave-one-category-out evaluation procedure. For each experiment, one of the five task categories was excluded entirely from training and validation. The remaining four categories were used to train and validate the classifiers. The held-out category was then used as the test set.

For each experiment:

Partition	Prompts	Responses
Training	704	2,112
Validation	176	528
Held-out test category	220	660

I trained separate CNN and LSTM models for each held-out category, using response-only input.

Results

Held-out category	CNN accuracy	LSTM accuracy
Creative Writing	73.6%	40.8%
General Knowledge	**92.4%**	**58.5%**
Programming	75.9%	56.2%
Mathematical/Logical Reasoning	75.2%	57.4%
Text Transformation	57.3%	47.3%

Average	**74.9%**	**52.0%**

CNN outperformed LSTM in all five held-out category experiments. General Knowledge had the highest CNN accuracy, at 92.4%. Text Transformation had the lowest, at 57.3%. Since each test category contained the same number of responses, the average accuracy represents an equally weighted summary of the five experiments. Importantly, the category results came from five separately trained classifiers, not one classifier tested across all categories.

Category-Specific Generalization

The results demonstrate that authorship-related patterns are not equally transferable across all writing tasks. For example, in the General Knowledge experiment, CNN correctly classified 97.3% of Gemini responses. However, when Creative Writing was held out, Gemini recall dropped to 31.4%. This suggests that some distinctive characteristics of Gemini's responses are more apparent in explanatory tasks than in creative writing. Text Transformation was another challenging category. CNN achieved approximately 95.1% validation accuracy on the four training categories but only 57.3% accuracy on the held-out Text Transformation test set. This large difference demonstrates a generalization gap between familiar and unfamiliar task categories. These observations also support the motivation behind selecting multiple categories. As discussed in the dataset design section, previous studies have emphasized the difficulty of detecting and attributing machine-generated text when the domain or prompting style changes. My RQ3 experiments investigated this issue directly by testing each category without including any examples from that category during training.

Comparison with the Pilot Study

Architecture	Pilot average	RQ3 accuracy	Expanded average	Change
CNN	65.0%	74.9%	+9.9 percentage points
LSTM	36.0%	52.0%	+16.0 percentage points

Both architectures improved compared with the pilot study. However, even with substantially more training data, unseen-category performance remained lower than the strongest standard classification results. This indicates that increasing the dataset size was associated with better performance, but it did not eliminate the difficulty of generalizing across different types of language-generation tasks.

Experiment 4: Linguistic Fingerprint Analysis (RQ4)

Experimental Design

The fourth experiment examined observable linguistic characteristics of the 3,300 generated responses. Unlike the previous experiments, this analysis did not train a neural network. Instead, I calculated descriptive features, including:

- Response length in words

- Type-token ratio (TTR), calculated as the number of unique lowercased words divided by the total number of lowercased words
- Average words per sentence
- Presence of Markdown headings
- Presence of bullet lists
- Presence of bold formatting
- Presence of code blocks

For TTR, punctuation and Markdown formatting tokens were excluded from the word counts. This measurement therefore reflects the ratio of unique words to total words rather than all tokenizer tokens. I also compared word counts for responses to the same prompt, allowing me to control for differences in prompt content when examining length.

Overall Linguistic Characteristics

Feature	GPT	Claude	Gemini
Mean word count	160.3	340.7	423.3
Median word count	110.0	317.0	404.0
Mean TTR	0.660	0.561	0.528
Mean words per sentence	14.4	14.9	15.3
Markdown headings	8.6%	10.9%	54.5%
Bullet lists	19.4%	36.0%	55.6%
Bold formatting	50.5%	69.4%	76.0%
Code blocks	15.6%	16.8%	16.2%

The most obvious differences involved response length and formatting. GPT produced the shortest responses on average, at approximately 160 words. Claude averaged approximately 341 words, while Gemini averaged approximately 423 words. Gemini also used Markdown headings much more frequently than either GPT or Claude. In contrast, the three models had relatively similar overall code-block usage rates.

Matched-Prompt Length Comparison

Because all three LLMs answered the same prompts, I compared their response lengths directly. Across the 1,100 matched prompts:

- GPT responses averaged 180.4 fewer words than Claude responses.
- GPT responses averaged 263.0 fewer words than Gemini responses.
- Claude responses averaged 82.7 fewer words than Gemini responses.

GPT produced a longer response than Claude on only 6.5% of prompts and a longer response than Gemini on only 5.1%.

These findings show that response length differences were not limited to one particular category or group of questions.

Formatting Differences Across Categories

The formatting patterns were strongly dependent on task category. For example, the percentage of Gemini responses containing Markdown headings was:

Task category	Gemini heading rate
General Knowledge	95.5%
Creative Writing	4.5%
Programming	99.5%
Mathematical/Logical Reasoning	68.6%
Text Transformation	4.5%

This pattern offers a possible explanation for some of the RQ3 results. In General Knowledge and Programming, Gemini often produced highly structured responses with headings and lists. Those patterns may make its outputs more recognizable. In Creative Writing, Gemini rarely used headings, making its responses structurally more similar to conventional prose. This interpretation is consistent with Gemini's lower CNN recall in the held-out Creative Writing experiment. However, these descriptive results do not prove that the neural networks relied on heading frequency or any other particular feature.

Type-Token Ratio Interpretation

GPT had the highest average type-token ratio, followed by Claude and Gemini. At first glance, this might suggest that GPT uses more diverse vocabulary. However, TTR is affected by text length: shorter texts often have higher TTR values because fewer words are repeated. Because GPT generated considerably shorter responses, I cannot interpret its higher TTR as evidence of greater vocabulary richness without controlling for response length. I therefore treat TTR as a descriptive measurement rather than a definitive comparison of linguistic sophistication.

Discussion

Which Architecture Performed Better? Across the experiments, CNN consistently outperformed LSTM. In the standard output-only experiment, CNN achieved 86.7% accuracy compared with 65.3% for LSTM. In the unseen-category experiments, CNN achieved 74.9% average accuracy compared with 52.0% for LSTM. These findings suggest that the CNN architecture was more effective for this particular dataset and experimental configuration. One possible explanation is that LLM-specific writing patterns often appear in local sequences, formatting conventions, and commonly used phrases that convolutional filters can detect. Nevertheless, I did not directly analyze internal model representations, so I cannot determine the exact features responsible for the CNN's advantage.

Did Increasing Dataset Size Help?

Both CNN and LSTM achieved higher test accuracy after expanding the dataset from 300 to 3,300 responses. The LSTM benefited particularly strongly, improving from 40.0% to 65.3% in the output-only experiment. These results suggest that the original dataset may have been too small for reliable sequence-model training. However, because the expanded experiment also used a different test set, further controlled experiments would be required to isolate the effect of training-set size.

Are LLMs Identifiable Across Different Tasks?

The RQ3 results suggest that LLM authorship can be identified in previously unseen categories, but performance varies significantly. CNN exceeded chance-level accuracy in all five categories, indicating that some model-specific patterns transfer between tasks. However, the relatively low accuracy on Text Transformation demonstrates that cross-category generalization remains challenging. Consequently, a classifier that performs well on familiar prompt categories should not automatically be assumed to perform equally well on all types of generated text. This is consistent with the motivation from M4GT-Bench, OpenTuringBench, and the research by Beigi et al. (2024), which emphasize the importance of evaluating machine-generated text identification across different domains and prompting conditions.

Relationship Between Classification and Linguistic Fingerprints

The linguistic analysis showed that GPT, Claude, and Gemini differ in response length, Markdown usage, and other formatting characteristics. These patterns provide plausible sources of information that an authorship classifier might exploit. For example, Gemini's frequent heading usage in technical and explanatory tasks may make its responses easier to distinguish from those of GPT and Claude. Nevertheless, the experiments establish an association between observable writing patterns and classification performance, not a causal explanation of the classifiers' decisions. An additional feature-ablation experiment would be necessary to determine whether removing Markdown formatting or standardizing response lengths reduces classification accuracy.

Limitations

There are several limitations to this project. First, the experiments included only three LLM configurations. The results may not generalize to different models or versions from the same providers. Second, the results may be affected by the specific prompting and generation settings used during data collection. Changing output-length limits, model parameters, or instructions could change the writing patterns observed in the dataset. Third, the models processed a maximum of 512 input tokens. Longer responses were truncated, which may have removed useful linguistic information. Fourth, the experiments mainly used one random seed and one training run per configuration. Different random seeds or training runs could produce somewhat different results. PyTorch also reported that some operations on the MPS backend may not be completely deterministic. Finally, the results do not prove which linguistic features were most important for classification. Although the RQ4 analysis identified differences in response length and formatting, further experiments would be needed to determine how much each feature contributed to CNN or LSTM performance.

Conclusion

This project investigated whether neural-network classifiers could identify the source of text generated by three Large Language Models. Using 3,300 responses from GPT, Claude, and Gemini, I compared CNN and LSTM architectures under multiple evaluation settings. The CNN achieved the strongest performance, reaching 86.7% accuracy using responses alone and 88.7% when combining prompts with responses. The LSTM also performed above chance, but its accuracy was consistently lower than the CNN's.

The cross-category experiments demonstrated that authorship identification can generalize to unfamiliar types of prompts, with CNN achieving 74.9% average accuracy. However, performance varied considerably across categories.

The linguistic fingerprint analysis also revealed differences in response length, Markdown formatting, and writing structure among the three LLMs.

Overall, the experiments suggest that LLM-generated text contains detectable model-specific characteristics. At the same time, the results highlight that identification accuracy depends on the task category, amount of available data, and classifier architecture.

Future work could explore additional LLMs, different generation settings, longer input sequences, multiple training seeds, and feature-ablation experiments to determine which linguistic characteristics contribute most strongly to classification performance.

References

Beigi, A., Tan, Z., Mudiam, N., Chen, C., Shu, K., & Liu, H. (2024). *Model Attribution in LLM-Generated Disinformation: A Domain Generalization Approach with Supervised Contrastive Learning.* arXiv. https://arxiv.org/abs/2407.21264